Molecular dynamics: 2D Lennard-Jones Simulation

Plabon Shaha and Guillem Masdemont
University of Ljubljana, Spring Semester, EMAI Cohort 25–27

May 5, 2026

Implementation We parallelize the algorithm and present several versions that are built on top of each other:

V1 — CPU baseline
- $O(n^2)$ nested loop
- Each pair counted twice
- Single CPU core

Baseline: ≈ 0.08× vs V2

V2 — OpenMP CPU
- $O(n^2)$, unique pairs only
- Newton's 3rd law
- Multi-core via OpenMP

Reference: 1×

V3 — GPU 2D grid
- GPU forces, 2D grid
- One thread per (i, j)
- Per-step malloc/free

≈ 4× vs V2

V4 — GPU 1D pairs
- 1D kernel, unique pairs
- Newton's 3rd law on GPU
- Integration on CPU

≈ 4.5× vs V2

V5 — Fully GPU, AoS
- All steps on GPU
- Single malloc/free
- If/else instead of fmod

≈ 5× vs V2

V6 — Fully GPU, SoA
- SoA layout
- Coalesced warp access
- r^2 cutoff check

≈ 5.4× vs V2

V7 — Neighbour list
- Verlet list, breaks $O(n^2)$
- Forces on nearby only
- Rebuild on drift

≈ 7.6× vs V2

V8 — 2-GPU naive
- Particles split across 2 GPUs
- P2P position exchange
- Full GPU integration

≈ 3× vs V2

Table 1: Summary of all versions. Speedups at $N = 1000$ steps, relative to V2 (OpenMP CPU).

Evaluation: We first analyze the performance across diferent version for different number of particles (see Figure 1), we plot logaritmic execution time scale.

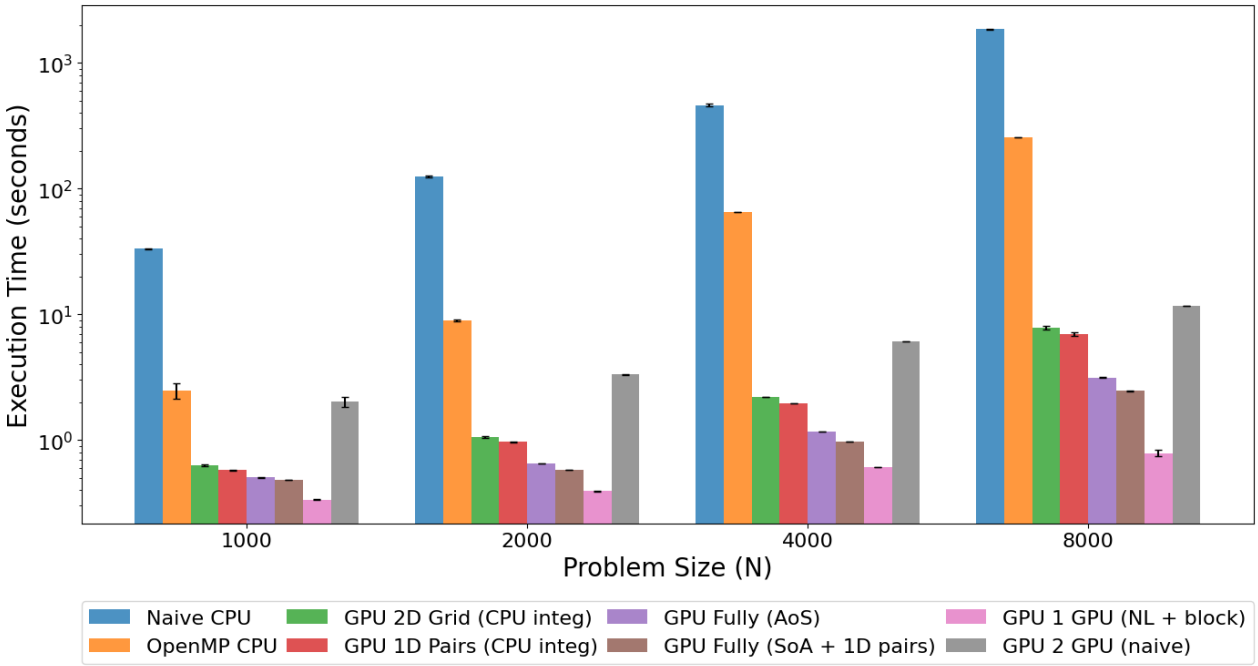

Figure 1: Performance comparison of the aforementioned CPU and CUDA optimization layers.

As shown in class, we see that the Naive CPU and OpenMP CPU baselines (blue-orange) scale as $O(n^2)$, going from N=1000 to N=8000 increases OpenMP CPU time by roughly 128× (from

2.5s to 250s), consistent with squaring the particle count. The GPU versions all show much shallower scaling. gpu_v2 and gpu_v3 (green and red) improve over CPU but still carry significant per-step transfer overhead since they allocate and free GPU memory every step, at N=8000 they reach 8s (about 30× faster than CPU). gpu_v4 and gpu_v5 (purple and brown) eliminate this overhead by keeping data resident on the GPU, bringing $N = 8000$ down to $2 - 3s$ and achieving roughly $80 - 100\times$ speedup over CPU. The neighbour list version gpu_v7 (pink) is the clear winner at all particle counts, at $N = 8000$ it reaches 0.8s, nearly 300× faster than CPU, because it breaks the $O(n^2)$ wall by limiting each particle to 40 neighbours rather than 8000. The 2-GPU version gpu_v8 (grey) is slower than gpu_v7 at all N, because the position exchange between devices at each step introduces synchronisation overhead that outweighs the benefit of splitting the work, at $N = 1000$ and $N = 2000$ the problem is simply too small to amortise that cost, and at $N = 8000$ the exchange still costs more than the neighbour list saves, although more close. We believe that combining the 2 GPU version with the V7 - Neighbour list, will not improve the best result on one GPU due to the memory transfer overhead.

Now we present the performance of V5 - Fully GPU, AoS version for different block sizes:

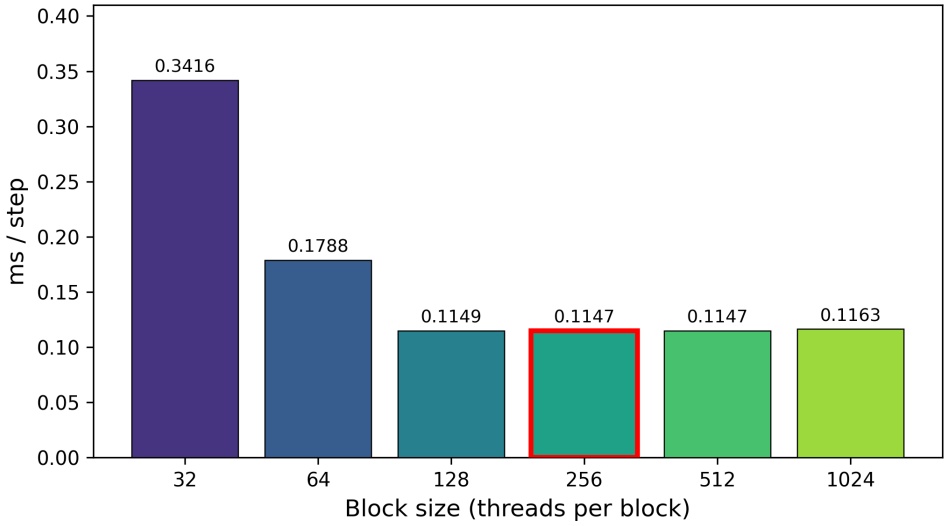

Figure 2: Block size tuning results for the V5 force kernel (N = 4000).

The sweep shows a clear performance cliff at small block sizes — block=32 takes 0.3416 ms/step, roughly 3× slower than the optimum. This is because 32 threads is exactly one warp, giving the scheduler no room to hide memory latency: when a thread stalls waiting for a global memory read, the entire block stalls with nothing to switch to. Doubling to block=64 already halves the time to 0.1788 ms/step, and block=128 drops further to 0.1149 ms/step as latency hiding improves with more warps in flight per block. Beyond 128 the curve flattens — 256, 512, and 128 all measure within 0.0002 ms/step of each other (0.1147–0.1149), meaning the GPU is fully saturated and additional warps per block provide no further benefit. Block=1024 slightly regresses to 0.1163 ms/step, likely due to increased register pressure reducing the number of blocks that can reside simultaneously on each streaming multiprocessor. In practice, blocksizes of 128 to 512 are optimal. In this scenario, we only measured the performance on computing the force, but since force kernel dominates runtime, the chosen block size is effectively optimal for the full leapfrog step.